

BAB 6 FIREBASE CLOUD MESSAGING (FCM)

6.1 Latar Belakang

Firestore Cloud Messaging (FCM) adalah solusi pengiriman pesan lintas platform yang memungkinkan Anda mengirimkan pesan dengan tepercaya tanpa biaya. Dengan menggunakan FCM, Anda bisa mengirim pesan secara masif pada suatu kelompok atau individu. Selain itu dengan FCM anda bisa mengirim data pada aplikasi mobile yang sudah ditentukan. Untuk kasus penggunaan pengiriman pesan instan, pesan dapat mentransfer payload hingga 4 KB ke aplikasi klien.

6.1.1 Kemampuan Firestore Cloud Messaging

Mengirim pesan notification atau pesan data	Mengirim pesan notification yang ditampilkan kepada pengguna. Atau mengirim pesan data dan menentukan sepenuhnya apa yang terjadi dalam kode aplikasi
Penargetan pesan serbaguna	Mendistribusikan pesan ke aplikasi klien dengan salah satu dari tiga cara : ke satu perangkat, ke grup perangkat, atau ke perangkat yang berlangganan topik.
Mengirim pesan dari aplikasi klien	Mengirim notification, chat, dan pesan lain dari perangkat ke server melalui saluran koneksi FCM

6.1.2 Cara Kerja Firestore Cloud Messaging

Implementasi FCM mencakup dua komponen utama untuk mengirim dan menerima pesan:

- Lingkungan tepercaya seperti Cloud Functions untuk Firestore atau server aplikasi yang akan digunakan untuk membuat, menargetkan, dan mengirim pesan.
- Aplikasi klien iOS, Android, atau Web (JavaScript) yang menerima pesan.

Anda dapat mengirim pesan melalui Admin SDK atau API HTTP dan XMPP. Anda juga dapat menggunakan Notifications Composer untuk menguji atau mengirim pesan pemasaran atau interaksi dengan penargetan dan analisis bawaan yang andal. Alur pengiriman pesan melalui FCM ditunjukkan oleh Gambar 6.1



Gambar 6.1 Alur pengiriman pesan menggunakan FCM

6.1.3 Alur Implementasi Firebase Cloud Messaging

1. Menyiapkan FCM SDK	Menyiapkan Firebase dan FCM pada aplikasi sesuai petunjuk penyiapan untuk platform Anda.
2. Mengembangkan aplikasi klien	Menambahkan penanganan pesan, logika langganan topik, atau fitur opsional lain ke aplikasi klien Anda. Selama pengembangan, Anda dapat mengirim pesan pengujian dengan mudah dari Notifications Composer
3. Mengembangkan server aplikasi	Tentukan apakah Anda ingin menggunakan Admin SDK atau salah satu protokol server untuk membuat logika pengiriman, yaitu logika untuk mengautentikasi, membuat permintaan pengiriman, menangani respons, dan sebagainya. Kemudian, buat logika di lingkungan yang tepercaya.

6.2 Jenis Pesan pada Firebase Cloud Messaging

Dengan FCM, Anda dapat mengirim 2 jenis pesan ke klien:

- Notification message, terkadang dianggap sebagai "pesan tampilan".
- Data message, yang ditangani oleh aplikasi klien.

Pesan notification memiliki serangkaian kunci bawaan yang terlihat oleh pengguna. Sebaliknya, pesan data hanya memuat *key-value pair* khusus. Berikut contoh dan penjelasan masing-masing jenis pesan di FCM

6.2.1 Notification Message

Untuk pengujian atau pemasaran dan penarik interaksi pengguna, Anda dapat mengirim pesan *notification* menggunakan Notifications Composer. Untuk mengirim pesan notification secara terprogram menggunakan Admin SDK atau protokol FCM, setel kunci notification dengan rangkaian opsi key-value bawaan yang diperlukan untuk bagian pesan notification yang terlihat oleh pengguna. Misalnya, berikut ini pesan notification berformat JSON dalam aplikasi IM. Pengguna akan menemukan pesan dengan judul "Pesan untuk anda" dan teks "Selamat pagi dunia" di perangkat:

```
{
  "to" : "/topics/INFO_DISKON_DAN_PROMO",
  "notification" : {
    "body" : "Dapatkan diskon 50% di toko kesayangan kalian",
    "title" : "Promo Diskon 50%"
  }
}
```

6.2.2 Data Message

Setel kunci data dengan *key-value pair* khusus Anda untuk mengirim payload data ke aplikasi klien. Pesan data dapat memiliki payload maksimal 4 KB. Misalnya, berikut adalah pesan berformat JSON dalam aplikasi instant messaging yang mengirimkan data (payload) yang diharapkan dapat diproses oleh aplikasi klien :

```
{
  "to" : "/topics/ INFO_DISKON_DAN_PROMO",
  "data" : {
    "promo" : "Dapatkan diskon 50% di toko kesayangan kalian ",
    "promold" : "P6712",
    "promoUntil" : "Berlaku hingga 10 tahun kedepan"
  }
}
```

6.2.3 Pesan Gabungan Notification dan Data

Baik secara terprogram atau melalui Firebase console, Anda dapat mengirim pesan notification yang berisi payload opsional key-value pair khusus. Di penulis Notifications, gunakan kolom Data khusus pada Opsi lanjutan. Perilaku aplikasi saat menerima pesan yang memuat payload notification dan data bergantung pada apakah aplikasi itu berjalan di latar belakang atau latar depan - intinya, apakah aplikasi aktif atau tidak pada saat menerima pesan.

- Saat berjalan di latar belakang, aplikasi menerima payload notification di baki notification, dan hanya menangani payload data saat pengguna klik pada notification yang masuk.
- Saat berjalan di latar depan, aplikasi Anda menerima objek pesan dengan kedua payload yang tersedia.

Berikut adalah pesan berformat JSON yang memuat kunci notification dan juga kunci data:

```
{
  "to" : "/topics/ INFO_DISKON_DAN_PROMO",
  "notification" : {
    "body" : "Dapatkan diskon 50% di toko kesayangan kalian",
    "title" : " Promo Diskon 50%"
  },
  "data" : {
    "promo" : "Dapatkan diskon 50% di toko kesayangan kalian ",
    "promold" : "P6712",
    "promoUntil" : "Berlaku hingga 10 tahun kedepan"
  }
}
```

6.3 Praktik Pembuatan Aplikasi

1. Daftarkan **nama package** dari project aplikasi anda ke Google Firebase
2. Download file **google-services.json** ke dalam project anda
3. Tuliskan kode program berikut ini

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

  <application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportRtl="true"
    android:theme="@style/Theme.Appx0f"
    tools:targetApi="31">

    <meta-data
      android:name="com.google.firebase.messaging.default_notification_channel_id"
      android:value="INFO_DISKON_DAN_PROMO" />

    <meta-data
      android:name="com.google.firebase.messaging.default_notification_color"
```

```

        android:resource="@android:color/system_accent2_10" />

        <meta-data
            android:name="com.google.firebase.messaging.default_notification_icon"
            android:resource="@android:drawable/ic_dialog_email" />

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service
            android:name=".MyFirebaseMessagingService"
            android:exported="false">
            <intent-filter>
                <action android:name="com.google.firebase.MESSAGING_EVENT" />
            </intent-filter>
        </service>

    </application>
</manifest>

```

build.gradle.kts (Project)

```

// Top-level build file where you can add configuration options common to all sub-
// projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.google.gms.google.services) apply false
}

```

build.gradle.kts (Module)

```

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.google.gms.google.services)
}

android {
    namespace = "polinema.mi.appx0f"
    compileSdk = 36

    defaultConfig {
        applicationId = "polinema.mi.appx0f"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
}

```

```

buildTypes {
    release {
        isMinifyEnabled = false
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}

buildFeatures {
    viewBinding = true
}

compileOptions {
    sourceCompatibility = JavaVersion.VERSION_21
    targetCompatibility = JavaVersion.VERSION_21
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.activity)
    implementation(libs.androidx.constraintlayout)
    implementation(libs.firebase.messaging)
    implementation(libs.androidx.work.runtime.ktx)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
}

```

libs.versions.toml

```

[versions]
agp = "8.13.0"
kotlin = "2.2.20"
coreKtx = "1.17.0"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
appcompat = "1.7.1"
material = "1.13.0"
activity = "1.11.0"
constraintlayout = "2.2.1"
googleGmsGoogleServices = "4.4.3"
firebaseMessaging = "25.0.1"
workRuntimeKtx = "2.10.5"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }

```

```

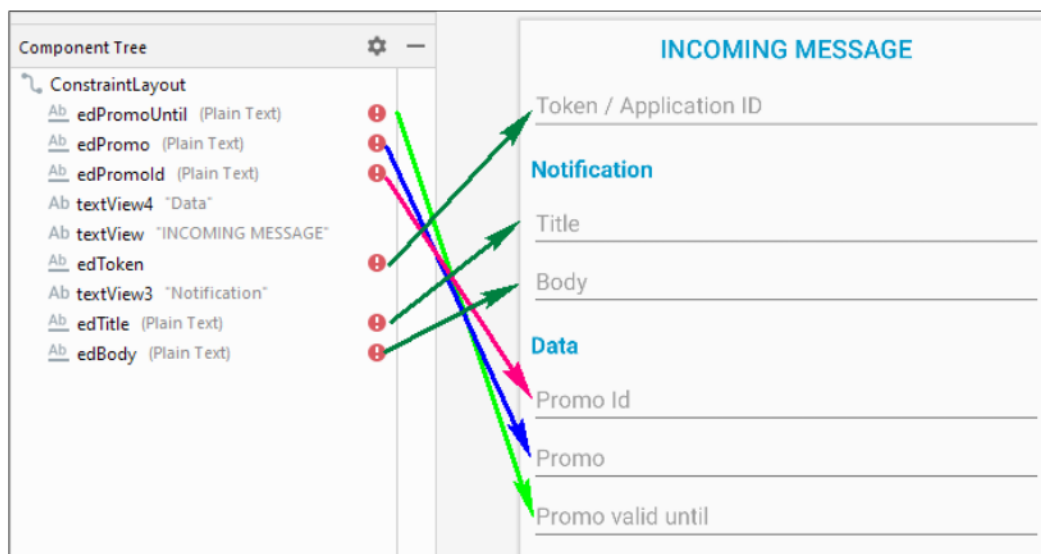
material = { group = "com.google.android.material", name = "material", version.ref =
"material" }
androidx-activity = { group = "androidx.activity", name = "activity", version.ref = "activity" }
androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout",
version.ref = "constraintlayout" }
firebase-messaging = { group = "com.google.firebase", name = "firebase-messaging",
version.ref = "firebaseMessaging" }
androidx-work-runtime-ktx = { group = "androidx.work", name = "work-runtime-ktx",
version.ref = "workRuntimeKtx" }

[plugins]
android-application = { id = "com.android.application", version.ref = "app" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
google-gms-google-services = { id = "com.google.gms.google-services", version.ref =
"googleGmsGoogleServices" }

```

Membuat antarmuka aplikasi

Antarmuka aplikasi (**activity_main.xml**) berguna untuk menampilkan pesan berupa notifikasi atau data. Rancangan antarmuka pengguna dapat dilihat pada Gambar 6.3



Gambar 6.2 Rancangan antarmuka pengguna (activity_main.xml)

MainActivity.kt

Class ini digunakan untuk menampilkan pesan/message yang masuk, baik pesan berupa notifikasi maupun data, dan dapat digunakan untuk memproses pesan lebih lanjut.

```

package polinema.mi.appx0f

import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import android.util.Log
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.core.view.ViewCompat

```

```

import androidx.core.view.WindowInsetsCompat
import com.google.firebase.Firebase
import com.google.firebase.messaging.messaging
import android.Manifest
import polinema.mi.appx0f.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    lateinit var b : ActivityMainBinding
    val TOPIC = "INFO_DISKON_DAN_PROMO" //topic yang akan di subscribe
    val TAG = "MainActivity"
    var bundle : Bundle? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        b = ActivityMainBinding.inflate(layoutInflater)
        setContentView(b.root)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(b.main.id)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }

    override fun onResume() {
        super.onResume()

        //versi Android Oreo keatas membutuhkan channel untuk menampilkan notifikasi
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val notificationManager = getSystemService(NotificationManager::class.java)
            notificationManager?.createNotificationChannel(
                NotificationChannel(TOPIC, TOPIC,
                    NotificationManager.IMPORTANCE_HIGH,
                )
            )
        }

        //meminta izin notifikasi dari user khusus untuk android device > 33
        askNotificationPermission()

        //berlangganan ke topik tertentu untuk mendapatkan notifikasi dari topik tersebut
        Firebase.messaging.subscribeToTopic(TOPIC)
            .addOnCompleteListener { task ->
                var msg = "Berhasil subscribe"
                if (!task.isSuccessful) msg = "Tidak berhasil subscribe"
                Log.d(TAG, msg)
                Toast.makeText(this@MainActivity, msg, Toast.LENGTH_SHORT).show()
            }

        //meminta token sebagai identifier device yang terhubung ke FCM
        Firebase.messaging.token.addOnCompleteListener{
            if (!it.isSuccessful) return@addOnCompleteListener
            val token = it.result
            Log.d(TOPIC, "Token = $token")
            b.edToken.setText("$token")
        }

        //cek kiriman data (bila ada)
    }
}

```

```

try {
    bundle = intent.extras!!
} catch (e: Exception) {
    Log.e(TAG, "${e.message}")
}

if(bundle!=null) {
    val keys = bundle!!.keySet()
    for(key in keys)
        Log.d(TAG, "$key : ${bundle!!.getString(key)}")
    b.edPromoId.setText(bundle!!.getString("promoId"))
    b.edPromo.setText(bundle!!.getString("promo"))
    b.edPromoUntil.setText(bundle!!.getString("promoUntil"))
    try {
        b.edTitle.setText(bundle!!.getString("title"))
        b.edBody.setText(bundle!!.getString("body"))
    } catch (e:Exception){
        Log.e(TAG, "${e.message}")
    }
}
}

private fun askNotificationPermission() {
    // This is only necessary for API Level > 33 (TIRAMISU)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(this,
            Manifest.permission.POST_NOTIFICATIONS) !=
                PackageManager.PERMISSION_GRANTED)
            { requestPermissionLauncher.launch(
                Manifest.permission.POST_NOTIFICATIONS) }
    }
}

private val requestPermissionLauncher = registerForActivityResult(
    ActivityResultContracts.RequestPermission()) { isGranted: Boolean ->
    if (isGranted) {
        Toast.makeText(this, "Izin notifikasi diberikan",
            Toast.LENGTH_SHORT).show()
    } else {
        Toast.makeText(this, "FCM tidak dapat berjalan tanpa izin notifikasi",
            Toast.LENGTH_LONG).show() }
    }
}
}

```

MyFirebaseMessagingService.kt

Class ini berfungsi sebagai service yang menerima pesan/message dan menampilkan notifikasi ke layar smartphone pengguna.

```

package polinema.mi.appx0f

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.graphics.BitmapFactory
import android.media.AudioAttributes
import android.media.RingtoneManager
import android.os.Build

```

```

import android.util.Log
import androidx.core.app.NotificationCompat
import com.google.firebase.messaging.FirebaseMessagingService
import com.google.firebase.messaging.RemoteMessage

class MyFirebaseMessagingService : FirebaseMessagingService() {
    val TAG = "FCMService"
    val TOPIC = "INFO_DISKON_DAN_PROMO"
    val RC_INTENT = 100
    var promoId = ""
    var promo = ""
    var promoUntil = ""
    var msgTitle = ""
    var msgBody = ""

    override fun onNewToken(token: String) {
        Log.d(TAG, "token baru = $token")
    }

    override fun onMessageReceived(message: RemoteMessage) {
        super.onMessageReceived(message)

        Log.d("msg", "msg = ${message.data}")

        val intent = Intent(this, MainActivity::class.java)
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP)

        //isi notifikasi
        msgTitle = message.notification!!.title!!
        msgBody = message.notification!!.body!!

        //jika notifikasi berisi data
        if (message.data.isNotEmpty()) {
            Log.d("msg", "msg = ${message}")

            promoId = message.data["promoId"]!!
            promo = message.data["promo"]!!
            promoUntil = message.data["promoUntil"]!!

            intent.putExtra("promoId", promoId)
            intent.putExtra("promo", promo)
            intent.putExtra("promoUntil", promoUntil)
            intent.putExtra("title", msgTitle)
            intent.putExtra("body", msgBody)
        }
        sendNotification(msgTitle, msgBody, intent)
    }

    //function untuk menampilkan notifikasi ke hp pengguna
    private fun sendNotification(title : String, messageBody: String, intent: Intent) {
        val pendingIntent = PendingIntent.getActivity(this, RC_INTENT, intent,
            PendingIntent.FLAG_ONE_SHOT or PendingIntent.FLAG_IMMUTABLE,
        )

        val defaultSoundUri =
            RingtoneManager.getDefaultUri(RingtoneManager.TYPE_RINGTONE)
        val audioAttributes = AudioAttributes.Builder().setUsage(
            AudioAttributes.USAGE_NOTIFICATION_RINGTONE)
            .setContentType(

```

```

        AudioAttributes.CONTENT_TYPE_SONIFICATION).build()
val notificationManager = getSystemService(Context.NOTIFICATION_SERVICE) as
    NotificationManager

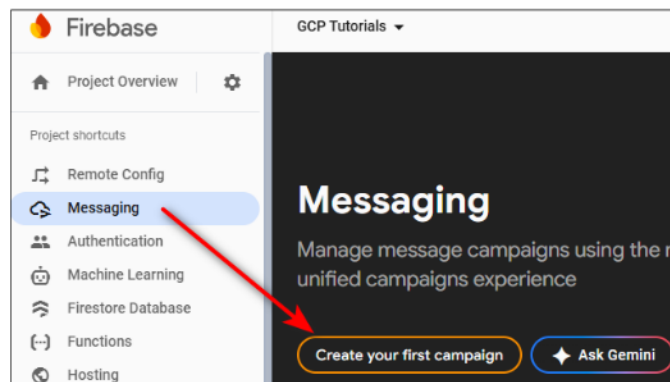
// Since android Oreo notification channel is needed.
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    val channel = NotificationChannel(
        TOPIC, TOPIC, NotificationManager.IMPORTANCE_HIGH)
    channel.description = TOPIC
    channel.setSound(defaultSoundUri, audioAttributes)
    notificationManager.createNotificationChannel(channel)
}

val notificationBuilder = NotificationCompat.Builder(this, TOPIC)
    .setSmallIcon(android.R.drawable.ic_dialog_info)
    .setLargeIcon(BitmapFactory.decodeResource(
        resources, android.R.drawable.ic_dialog_info))
    .setContentTitle(title)
    .setContentText(messageBody)
    .setSound(defaultSoundUri)
    .setContentIntent(pendingIntent)
    .setAutoCancel(true)
notificationManager.notify(RC_INTENT, notificationBuilder.build())
}
}

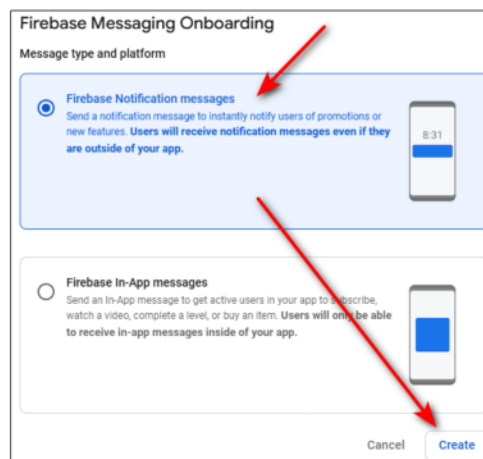
```

6.4 Uji Coba Aplikasi

1. Pada menu **Messaging**, klik pada tombol “**Create your first campaign**”



2. Klik pada **Firebase Notification messages**, dan kemudian klik **Create**



3. Untuk mengirim **notifikasi**, isikan informasi/teks pada menu berikut ini

1 Notification

Notification title ⓘ

Notification text
 ⓘ

Notification image (optional) ⓘ
 ⓘ

Notification name (optional) ⓘ
 ⓘ

Next

2 Target

User segment **Topic** ⓘ

Message topic ⓘ

Next

4 Key events (optional)

Sent ⓘ | Opened ⓘ |

Analytics Label ⓘ

Next

3 Scheduling

Send to eligible users

Next

5 Additional options (optional)

All fields optional

Android Notification Channel ⓘ

Custom data ⓘ

Key	Value

Sound

Expires ⓘ

Save as draft **Review**

4. Kemudian, klik tombol **publish** untuk mengirim **notification** pada *subscriber* channel **INFO_DISKON_DAN_PROMO**. Tunggu beberapa waktu untuk menerima notif pada HP anda

Review message

Notification content

 Dapatkan diskon 50% di toko kesayangan kalian

Target

 Subscribers of PML_FCM topic

Scheduling

 Send now

Cancel

Publish

5. Untuk menguji coba pengiriman **notifikasi** dan **data**, maka duplikasikanlah **campaign** yang tadi sudah dibuat dan tambahkan data-data berikut ini dan setelah itu klik **Publish**. Tunggu beberapa waktu, pengiriman akan berlangsung agak lama

Filter campaigns

Search by campaign name, description ...

New experiment

New campaign

Campaign	Start	End	Status	Target	Last Updated	Sends / Impressions	Clicks / Opens
Promo Diskon 50% Dapatkan diskon 50% di toko kesayangan kalian	Oct 1, 2025 11:35:57 PM	-	Active	PML_FCM	Oct 1, 2025	<1000	0%

0/10 Recurring notifications

Edit

Duplicate

Delete



5 Additional options (optional)

All fields optional

Android Notification Channel ?

INFO_DISKON_DAN_PROMO

Custom data ?

promo

Dapatkan diskon 50% di toko kesayangan kalian

promold

P6712

promoUntil

Berlaku hingga 10 tahun kedepan

Key

Value

Sound

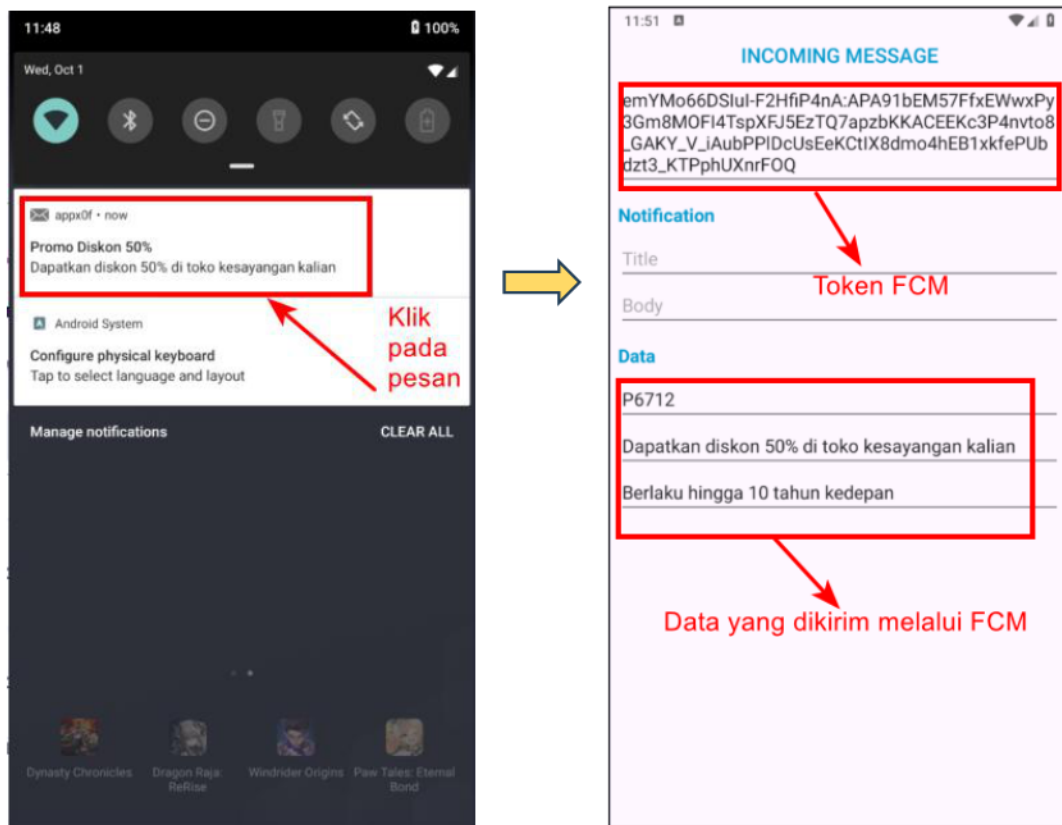
Disabled

Expires ?

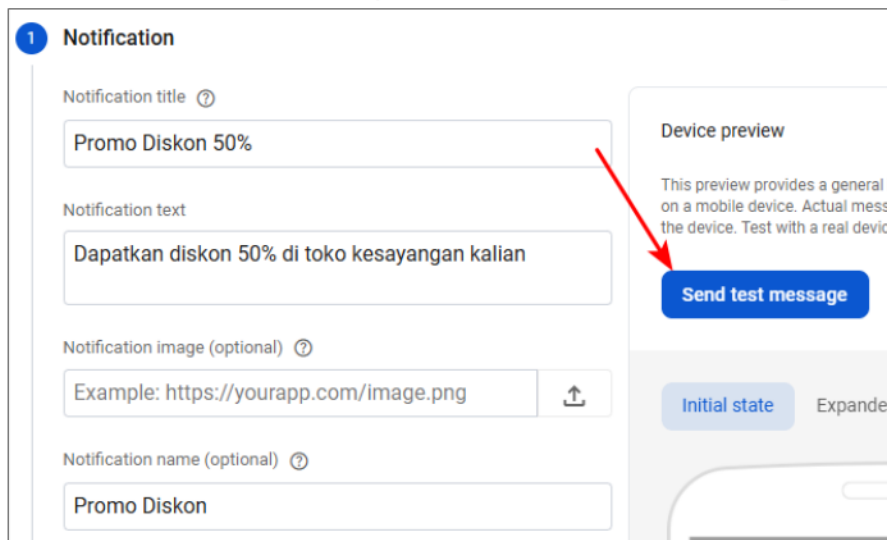
4

Weeks

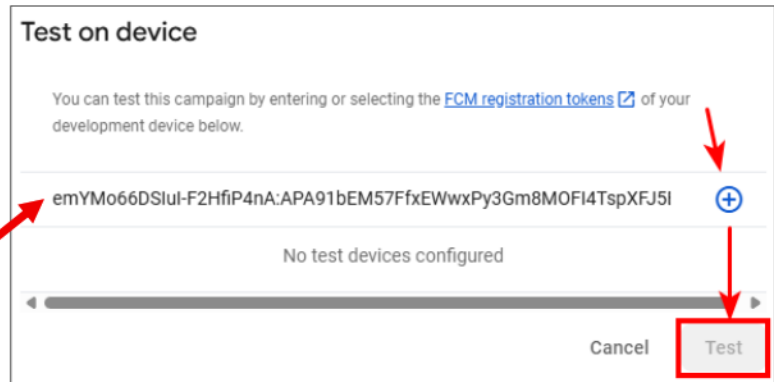
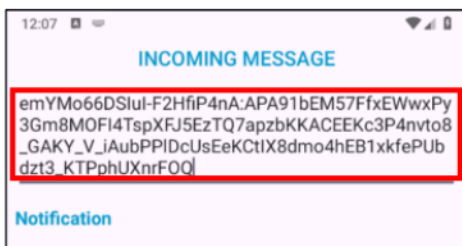
6. Setelah notifikasi muncul, maka klik pada notifikasi tersebut untuk menampilkan data yang masuk ke aplikasi Android anda



7. Untuk mengirim notification / data ke perangkat Android yang spesifik bisa dilakukan dengan menggunakan **token FCM**, caranya :
- Duplikasi campaign yang tadi sudah dibuat
 - Pada menu Notification, klik pada tombol **Send test message**



c. Salin **Token FCM** yang terdapat di HP ke **dashboard FCM** dan kemudian klik test



Referensi :

<https://firebase.google.com/docs/cloud-messaging/android/client>